

A Pen and a Paper in 3D Space

Computer Graphics & Animation Project Report

1. Project Requirements

The primary goal of this project is to implement a high-fidelity, interactive 3D scene of a writing workspace using **Three.js**. The scene features a piece of paper resting on a wooden desk with a pen drawing dynamically on it. The implementation successfully incorporates the following requirements:

- **Custom GLSL Shaders:** Custom vertex and fragment shaders to render complex marble and holographic surface finishes.
- **Lighting:** Focused local lighting (spotlight desk lamp) combined with ambient fill and localized tip lighting, including soft shadow-mapping support.
- **Perspective Projection:** Perspective viewport mapping with realistic focal settings.
- **Textures:** Custom procedural textures for the desk (walnut wood), paper (notebook ruled lines and pulpy fiber bumps), and pencil (cedar stripes).
- **Animations:** Cursive handwriting animation using spline tracking and parametric curves, paired with realistic pen lift/drop and directional tilt kinematics.
- **Interactivity:** Spherical camera orbiting via keyboard inputs, full orbital navigation via mouse, dynamic pen switching, color picking, and an interactive mouse-drawing canvas mode.

2. Software Platform

The project is built entirely on standard WebGL standards. The tools and technologies utilized include:

- **HTML5:** Structural layout and containers.
- **CSS3 (Vanilla):** Glassmorphic UI overlays, styles, and typography.
- **JavaScript (ES6 Modules):** Core application logic.
- **Three.js (r147):** WebGL 3D rendering wrapper.
- **OrbitControls Add-on:** Camera orbiting, panning, and zooming.
- **HTML5 Canvas API:** Programmatic procedural texture and bump map generation.

3. Project Features

3.1 Models and Texture Mapping

All 3D objects and meshes are built using Three.js geometries, combined with dynamic custom texture maps to optimize resource loading:

- **The Desk:** Built from a `BoxGeometry` and mapped with a procedural walnut wood texture. It calculates growth rings and wood fibers in an HTML5 canvas context.
- **The Paper:** Styled with A4 proportions. It uses a `BoxGeometry` with custom materials. The top face represents a dynamic canvas texture containing rules, red margins, and fiber noise, which acts as the active ink-drawing buffer.
- **The Pens:** Swappable pen models consisting of:
 - *Fountain Pen*: Tapered metal grip, calligraphy gold nib, and a marble shader barrel.
 - *Modern Stylus*: Holographic barrel, dark titanium accents, and a glowing LED tip ring.
 - *Wooden Pencil*: Hexagonal body with cedar stripes, shaved wood cone, and a graphite lead tip.
- **The Glass Holder:** A transmissive glass cup where unused pens dynamically rest.

3.2 Key Interaction

Keyboard interaction allows camera orbiting using spherical coordinate calculations:

- `W / ArrowUp` and `S / ArrowDown` adjust the polar angle (ϕ).
- `A / ArrowLeft` and `D / ArrowRight` adjust the azimuthal angle (θ).
- `Space` resets the camera back to the original isometric viewport angle using cubic easing interpolation.

3.3 Mouse Interaction

Mouse controls are context-aware:

- **Camera Orbiting:** Uses `OrbitControls` to orbit (left drag), pan (right drag), and zoom (scroll).
- **Interactive Canvas Drawing:** When "Enable Mouse Drawing" is toggled, mouse movements project a ray onto the paper. Dragging draws ink strokes directly onto the paper texture, and the 3D pen follows.
- **UI Panel Selection:** Buttons let the user swap pen types, colors, clear canvas, or pause auto-write.

3.4 Animation

The auto-write engine runs along a pre-compiled path of splines:

- The path plots out cursive letters spelling "Three.js" and finishes with a mathematical spirograph curve.
- The pen dynamically lifts off the paper, travels in the air, and drops back down to draw.
- The pen tilts dynamically opposite to its movement direction, simulating realistic hand-dragging kinematics.

3.5 Lighting

The workspace employs three light sources:

- **SpotLight:** A desk lamp light source ($0xffff1e0$, intensity 2.2) configured with `THREE.PCFSoftShadowMap` to cast soft, realistic pen shadows onto the paper.
- **AmbientLight:** Warm room fill lighting ($0xffffffff$, intensity 0.35).
- **PointLight:** Positioned at the pen tip. When the Modern Stylus is active, it emits neon light casting localized light reflections onto the paper.

3.6 Custom Shaders

Custom GLSL code is used for pen body materials:

- **Vertex Shader:** Transforms vertex coordinates, normal vectors, and UV coordinates, passing them to the fragment shaders.
- **Marble Fragment Shader:** Implements Fractional Brownian Motion (fBm) noise to generate organic marble veins, shaded via a Blinn-Phong specular lighting model.
- **Holographic Fragment Shader:** Uses the dot product of the surface normal and view direction (Fresnel angle) to shift the material color toward a rainbow gradient, creating a thin-film iridescent metal look.

3.7 Project Feature Status Table

#	Feature Name	Classification	Description
1	Custom Shaders	Implemented	Custom GLSL fBm marble noise and holographic Fresnel shaders.
2	Lighting & Shadows	Implemented	Soft shadows via SpotLight, ambient fill, and pen tip light.
3	Perspective Projection	Implemented	Perspective camera calibration and Target orbiting constraints.
4	Procedural Textures	Implemented	Desk wood grain, lined notebook page, and graphite lead.
5	Spline Animation	Implemented	Continuous cursive script and spirograph curves drawing.
6	Keyboard Camera Orbit	Implemented	Spherical orbiting using WASD/Arrows and Space reset.
7	Mouse Interaction	Implemented	Raycasting coordinate mapping to paint custom ink drawings.
8	Model Swapping UI	Implemented	Swappable Fountain Pen, Modern Stylus, and Wooden Pencil.
9	Interactive Pen Holder	Implemented	Glass holder that automatically updates containing unused pens.

Table 01: Project Feature Table

4. Snapshots

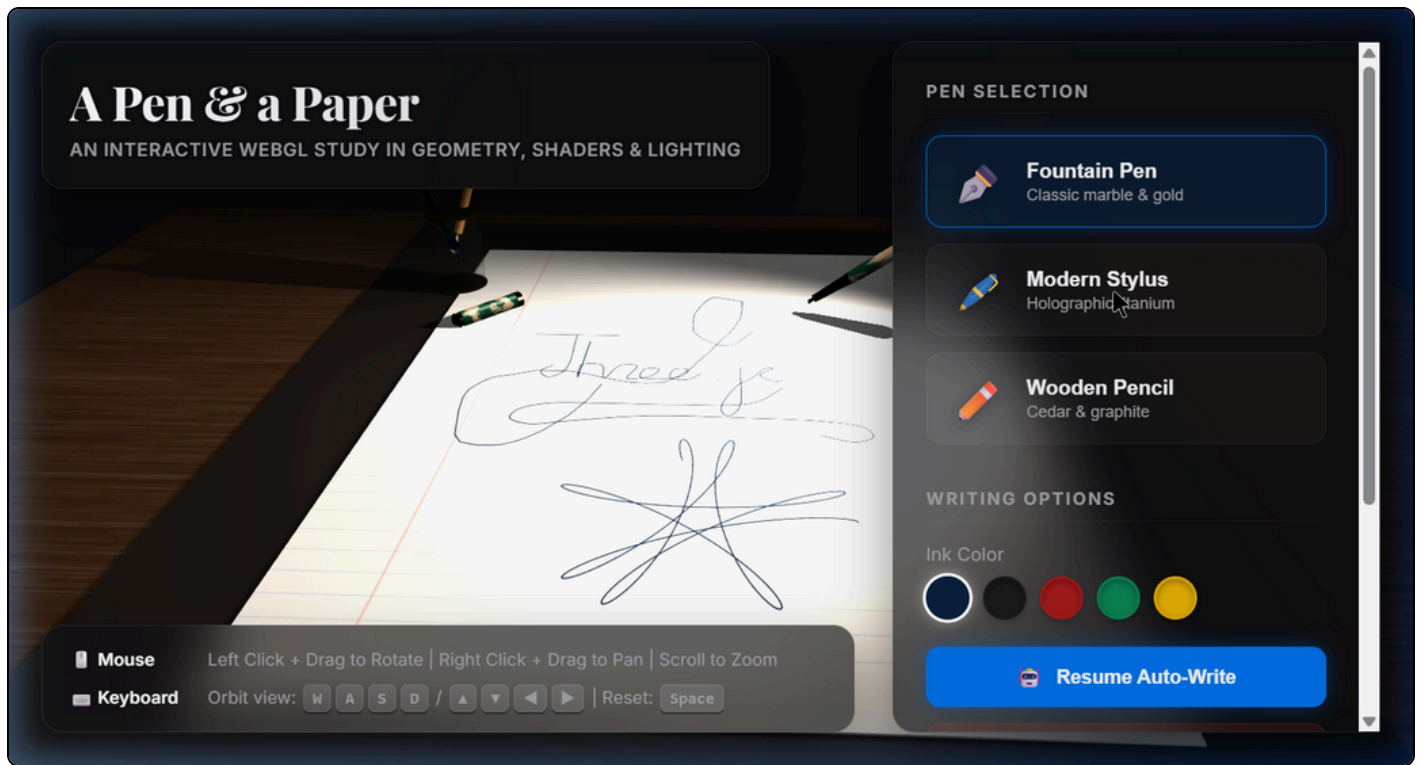


Figure 01: High-fidelity rendered 3D scene displaying writing path, desk lamp shadows, and marble shader.

5. Contribution

The project development tasks were distributed among the group members as follows:

Student Name	Student ID	Contribution & Tasks Performed
[Member 1 Name]	[Member 1 ID]	Shader implementations (vertex and fragment), mathematical spirograph equations, and keyboard orbiting camera physics.
[Member 2 Name]	[Member 2 ID]	Geometry modeling of the pens, desk lamp, glass holder, and UI overlay panel styling sheets.
[Member 3 Name]	[Member 3 ID]	Procedural canvas texture generators, raycasting drawing coordinates mapping, and spline path transitions.

Table 02: Contribution Table